TRAINING A POLICY GRADIENT AGENT ON CUSTOM PONG WITH PROGRESSIVE ANGLE MECHANICS

Author:	Korbinian Much
Date:	December 2025
Course:	Reinforcement Learning
Program:	M.Sc. Applied Artificial Intelligence
University:	IU International University of Applied Sciences

Table of Contents

1. Introduction **1**

1.1. Motivation 1

1.2. Key Challenges 1

1.3. Outline and Approach 2

2. Game Environment Description **3**

2.1. Overview 3

2.2. The Playing Field and Scoring System 4

2.3. Game Mechanics: Ball Dynamics and Paddle Movement 4

2.4. Opponent Behavior 4

2.5. Progressive Angle Mechanics 5

3. Literature Review **6**

3.1. Atari Games as an RL Benchmark 6

3.2. Conceptual Foundation: The REINFORCE Algorithm 6

3.3. Our Setting: A Feature-Based REINFORCE Agent 7

4. Training the Agent **8**

4.1. Conceptual Layout 8

4.2. The Setup 8

4.3. Results 9

5. Discussion **12**

5.1. Success in Exploration and Micro-Skill Acquisition 12

5.2. Failure in Context-Dependent Policy and Mastery 13

5.3. Comparison to Prior Work 13

6. Conclusion and Outlook **14**

6.1. Conclusion 14

6.2. Outlook 14

7. Bibliography **16**

8. Appendices **18**

8.1. Hyperparameters and Reward Shaping 18

8.2. Source Code (GitHub Link) 18

1. Introduction

1.1. Motivation

Atari 2600 games are popular in training and testing reinforcement learning models because they offer relatively complex problems while at the same time demanding relatively little computational power to solve them. "The Arcade Learning Environment (ALE) is a new evaluation platform for general agents... built on top of the Stella Atari 2600 emulator" (Bellemare et al., 2013, p. 255). One of the most popular games in the Atari catalogue is Pong, which is often chosen for RL experiments due to its simple mechanics (only 3 types of actions) yet balanced challenge.

In this paper we investigate whether a feature-based vanilla REINFORCE agent can learn to exploit the progressive angle mechanics in a custom Pong environment through reward shaping alone, when our agent plays against a rule-based computer opponent. We will not compare different algorithms and only use a feature based algorithm, but we will rather examine if our agent can consistently win against a rule-based opponent. For that we will adopt a policy gradient method for training. We implement a feature based REINFORCE algorithm introduced by Williams (1992) and train an agent using a custom Pong environment. Our focus lies on understanding how the agent develops its strategy over time. For that we will not just analyze final performance but describe different approaches and learned behavior. During training we are particularly interested in when and how our agent figures out how to exploit to hit the ball with the paddle edge and will make this a key focus in our observation. But first we will look at our key challenges and our approach.

1.2. Key Challenges

Pong differs from high-score games like Tetris, where feedback is constant and progress can be measured steadily by said high score. Pong, however, only knows two final states: Winning or losing after a hard score cap of 21 points, without feedback in-between. We will try to challenge the problem with a vanilla REINFORCE-method and systematic reward structure design. This setting is a documented challenge for policy gradient methods. Good initialization, however, and good hyperparameter adjustments should promise good results. We will highlight a few well-documented challenges that are characteristic of PG methods in sparse-reward, episodic environments:

Sparse Rewards: The agent receives feedback only when points are scored, which might (or might not) occur after hundreds of actions. This temporal credit assignment problem makes it

difficult to determine which earlier actions contributed to success or failure (Sutton & Barto, 2018).

High Variance in Returns: Episode outcomes vary a lot due to initial conditions and stochastic exploration. This variance in gradient estimates slows learning and requires many episodes to converge (Williams, 1992).

Long Episode Horizons: Games continue until win conditions are met. They have a long sequence of state-action pairs. The discounted return calculation is challenging because the rewards have to be propagated backward through potentially hundreds of time steps. This process is necessary to assign credit for actions taken early in the long episodes (Sutton & Barto, 2018).

Sensitivity to Initialization: Policy gradient methods are highly sensitive to initial weights and hyperparameter choices (Sutton & Barto, 2018). Meaning poor initialization can lead to early convergence towards a failing policy or might even result in a complete failure to learn.

Sample Inefficiency and Slow Convergence: Policy gradient methods require many complete episodes to estimate gradients accurately. Unlike value-based methods with experience replay, REINFORCE discards trajectory data after each update (Sutton & Barto, 2018). This results in sample inefficiency, meaning converging to an effective policy can require thousands of episodes, making training time-intensive.

Reward Scale Sensitivity: The magnitude and balance of rewards affects the dynamic of learning in a critical way: Strong penalties can likely bring the agent to learn avoidance behavior rather than goal-seeking strategies (Ng, Harada, & Russell, 1999). Reward shaping must always find the right balance of primary objectives with signals to guide exploration without overwhelming the learning process.

Summary: These difficulties are exacerbated in our custom Pong environment because our progressive angle system creates a particularly hard exploration problem: the optimal edge-hitting strategy is both high-reward and unlikely to be discovered by chance (Sutton & Barto, 2018). So the sparse-reward and credit-assignment issues must be emphasized in this context.

1.3. Outline and Approach

Given the training instabilities and challenges identified, we adopt a reward shaping strategy built around strong positive reinforcement at multiple skill stages. For that we set up sophisticated reward settings combined with standard REINFORCE techniques. The core idea is to design a reward structure that guides the agent to acquire skills sequentially. We expect

that our approach will enable the agent to discover an edge-hitting strategy while at the same time eliminating potential training instability. Our minimal extension of vanilla REINFORCE provides a promising solution to policy gradient challenges in sparse-reward environments. We expect to see clear behavioral stages which will allow a detailed strategy analysis.

In the following we will describe our custom Pong environment and its progressive angle mechanics. Then will review policy gradient methods and prior work on Atari reinforcement learning. After that we will describe our REINFORCE agent, feature representation, reward function, and training setup. We will then evaluate our results on learning progress, edge-hitting behavior and match-level performance and finally conclude with a discussion of limitations and directions for future work.

2. Game Environment Description

2.1. Overview

The environment we use for training is a custom implementation of Atari 2600 Pong. Originally we did this due to compatibility issues with OpenAI gym, but later found that our setup would allow us full insight and control over game physics and the parameters for experiments. Also it is easier to predict and interpret an agent's behavior and formulate an hypothesis for training.

The environment exposes a low-dimensional observation vector instead of raw pixels. At each time step the agent observes a 16-dimensional feature vector comprising ball position (x, y), ball velocity (vx, vy), player and opponent paddle positions, relative vertical offsets between ball and paddles, and simple trajectory information (e.g., predicted intersection points with the paddles). All features are normalized to lie in fixed numeric ranges, which keeps the input scale stable for the neural network. The discrete action space consists of three actions: STAY (no movement), UP (move the player paddle up by 4 pixels), and DOWN (move the paddle down by 4 pixels). The core idea of Pong is based on the concept of table-tennis: Two players control vertical paddles on opposite sides of the screen and use them to hit a square ball back and forth. A player-controlled paddle is on the right side and a computer-controlled opponent on the left side. A point is scored when the ball passes an opponent's edge of the court. The objective is to score 21 points. It also features a constant ball speed without speed acceleration. The environment thus resembles the core mechanics of classic Pong on Atari.

2.2. The Playing field and Scoring System

The game takes place on a rectangular court measuring 160 × 192 pixels. The top and bottom boundaries act as walls that the ball bounces off from. The left and right edges serve as goal lines. Both paddles are positioned 10 pixels from the edges from the left and right edge respectively. When the ball goes past the left edge, the player scores one point. When the ball flies past the right edge, the CPU scores one point. After each point, the ball returns to the center of the court and is played towards the side of the player who scored a point, which offers the opportunity to regain momentum. Each serve begins with a random angle to prevent pattern exploitation. The game continues until one player reaches 21 points. That player wins the match. There is no time limit; games end only when the score condition is met.

2.3. Game Mechanics: Ball dynamics and Paddle movement

The game's physics work with discrete time steps: For each time step one action is allowed for the player and one for the opponent. Both paddles are 2 pixels wide and 12 pixels tall. The ball is a square of 1 × 1 pixels. The dimensions of the paddles and the ball are the same as the ones from the original Atari 2600 game. The ball travels at a constant velocity throughout the game. At the start of each point, the ball appears at the center of the court and launches at a random angle within a range where it can easily be played back. This prevents deterministic serve patterns and prevents the agent from overfitting.

Both paddles can move vertically along their side of the court, but they cannot go beyond the boundaries of the court. The player paddle (right side) responds to three actions: STAY (no action), UP (4 pixels up), or DOWN (4 pixels down).

The ball bounces off the top and bottom walls of the court and when it hits with a paddle, it bounces back towards the opposite side with an angle calculated through specific angle mechanics. But there is no spinning or other effects involved, which ensures that identical inputs produce identical outputs.

2.4. Opponent Behavior

The CPU opponent constantly tries to align the center of its paddle with the vertical position of the ball. But it has restrictions, because we want to make it still beatable: It only begins moving when the ball crosses the midline (center) of the court. Before the ball reaches the center, the CPU remains, where it is. Because of that the player has the opportunity to take advantage of situations when the opponent's paddle is out of position. Also the CPU only moves with a maximum speed of only 4 pixels per action (like the player). This means the opponent defends

well against shallow-angle shots, but does not defend well when the ball comes from a steep angle. These limitations make the opponent challenging but beatable for both a human player and an A.I. agent. A player who masters the progressive angle system can consistently score points and we expect a deep reinforcement learning A.I. to do the same.

2.5. Progressive Angle Mechanics

The distinctive feature in this environment is a progressive angle system: the closer to the paddle edge, the steeper the angle in which the ball bounces off the paddle. The ball velocity remains constant through the game. So a sophisticated player will try and make use of the progressive angle system to outplay the opponent.

For that the paddle is divided into 12 different zones across its 12-pixel height: The top zone (approximately pixels 0-3) produces steep upward angles, the middle zone (pixels 4-7) produces shallow angles, and the bottom zone (pixels 8-11) produces steep downward angles. Steep angles are harder to hit for the opponent, because it must move at a greater distance to reach the ball. If the opponent is poorly positioned, these shots should result in a score. Because of that we expect an A.I. to exploit “edge-hits” as a winning game strategy.

3. Literature Review

3.1. Atari Games as an RL Benchmark

The Arcade Learning Environment (ALE), introduced by Bellemare et al. (2013), made Atari 2600 games a standard benchmark for reinforcement learning including Atari Pong. One of the most influential deep RL methods, developed and tested with said benchmark, was the Deep Q-Network (DQN) introduced by Mnih et al. (2013, 2015, p. 529): "We demonstrate that the deep Q-network (DQN) can master a variety of Atari 2600 games, using only the raw pixels and the game score as inputs." It combines Q-learning with convolutional neural networks, experience replay, and a target network to learn value functions directly from raw pixel inputs. It reached human-level performance on many Atari games, and it usually solves Pong fairly quickly with enough exploration experience.

Value-based methods like DQN produce their policy more indirectly. They always choose the action with the highest estimated Q-value. Later policy-based and actor-critic methods were also applied to Atari. For example Asynchronous Advantage Actor-Critic (A3C) from Mnih et al. (2016) "presents an asynchronous actor-critic algorithm (...) that uses asynchronous gradient descent for optimization of deep neural network controllers." This method often matches or

beats DQN on many games. Implementations like dueling network architectures "improved the performance even more" (Wang et al., 2016).

To sum up: Research shows that with large convolutional networks, stabilizing techniques (like experience replay, target networks, critics, trust regions, or clipped objectives) and parallelism deep RL can perform well on Pong. However, these strong agents usually work with raw pixel inputs and rely on many complex components. Because of that, they don't tell us much about how a simple policy gradient method behaves when it uses hand-crafted features instead of raw pixels, especially in a controlled Pong version with specific mechanics like our progressive angle system. Practical reports suggest that "Policy gradient methods are known for their high variance and instability... They are very sensitive to the choice of hyperparameters" (Lapan, 2018). This is why we seek to study this simpler, more focused setting in more detail.

3.2. Conceptual Foundation: The REINFORCE Algorithm

Policy gradient methods directly improve the policy by following an estimate of the gradient of the expected total reward. Unlike value-based methods that first learn a value function and then pick the best action, policy gradients simply adjust the probability of each action based on how well it performs. In our Pong environment, if an action usually leads to higher total rewards, the policy learns to pick it more often. If it leads to lower rewards, the policy picks it less often.

We use the basic REINFORCE algorithm, which was introduced by Williams in 1992. REINFORCE is a Monte Carlo policy gradient method. "The method is called REINFORCE... It is a Monte Carlo method, in that it learns from complete episodes of experience" (Williams, 1992). It works by letting the agent play complete episodes, calculating the discounted return for every time step, and then updating the policy parameters. Actions that were followed by higher returns get their probabilities increased, while actions followed by lower returns get their probabilities decreased.

The policy gradient theorem, proved by Sutton et al. in 1999, gives the theoretical justification for these methods. "The policy gradient theorem provides a beautiful and simple formula for the gradient of the expected return with respect to the policy parameters" (Sutton et al., 1999). It shows that the gradient of the expected return can be estimated using only samples of states, actions, and rewards. This means we can use function approximators, like neural networks, to represent the policy.

Later research developed more advanced methods such as TRPO, GAE, and PPO (Schulman et al., 2015a; 2015b; 2017). "We propose a new family of policy gradient methods... called Proximal Policy Optimization (PPO)" (Schulman et al., 2017). In this project, however, we

deliberately chose the plain Monte Carlo REINFORCE algorithm without any extra features like a critic or trust regions. By keeping it simple, we can more clearly observe and understand the basic learning behavior of policy gradients in our Pong setup.

3.3. Our Setting: A Feature-Based REINFORCE Agent

With our setup we want to study what a simple policy gradient setup can learn in Pong, when it uses engineered features and a custom reward shaping. Our three key design choices follow directly from the literature and our research goals.

First, we decided to use a small, 16-dimensional feature-based neural policy instead of processing pixels. While modern deep RL popularized learning from raw visual input, standard feedforward networks continue to perform effectively when learning from low-dimensional feature inputs (Goodfellow et al., 2016). This way we keep the problem computationally light and it also makes the learned behavior (ball and paddle positions, velocities, and simple trajectory information) easy to interpret.

Second, we choose vanilla REINFORCE over more advanced algorithms such as A3C, TRPO, or PPO. These methods are more sample-efficient and they are robust on large benchmarks (Mnih et al., 2016; Schulman et al., 2017), but they also introduce additional components (critics, trust-region constraints, multiple workers etc.). That will make the relationship between reward signals and actions in the game less clear to interpret and our main goal is to understand what the agent is learning and why. So we decide for a trade-off: transparency for efficiency and a focus on understanding the specific learning dynamics of the policy gradient process. By choosing handcrafted features over pixels, we accept a loss in the "general-purpose" capability that end-to-end deep learning offers (Mnih et al., 2015), but we gain clear insight into exactly what information the policy is working with.

Third, we will use our custom Pong environment to examine how reward design interacts with the REINFORCE algorithm when rewards are sparse and delayed. Atari literature and general reinforcement learning theory both highlight that sparse rewards and long time horizons create "the problem of temporal credit assignment: which of the many actions taken during the episode was responsible for the final reward?" (Sutton & Barto, 2018, p. 58). By carefully designing the reward function using potential-based shaping (Ng et al., 1999), we can study these challenges more directly. Our hypothesis is that a well-structured reward function can stabilize our REINFORCE algorithm and enable a simple agent to discover and consistently use the progressive angle strategy. The experiments that follow will evaluate this hypothesis and will

analyze when and how the agent discovers edge-hitting and the systematic weaknesses of the learned policy.

4. Training the agent

4.1. Conceptual Layout

As laid out earlier, we define three conceptual stages of learning progress (milestone tasks) and expect the agent's behavior to progress through them sequentially:

Stage 1 (Defense): Tiny rewards for paddle hits to encourage the agent to return the ball.

Stage 2 (Scoring): A bigger reward for scoring to encourage an offensive play style.

Stage 3 (Winning Games): A huge reward for winning a match to encourage the agent to exploit the progressive angle system and seek to win matches fast.

We interpret learning progress in terms of these three conceptual stages, but do not change the reward function over time. The reward structure is designed so that the agent will first learn paddle defense, then scoring, and finally match-winning play.

4.2. The Setup

Our environment knows three actions (STAY, UP, DOWN). For our action evaluation we implemented a two layer policy network with 16 input features, a hidden layer with 128 ReLU units and a softmax output. For training we implemented the REINFORCE algorithm. It uses RMSprop with decay factor 0.99, learning rate 0.0015 and discount factor 0.99. Gradients were calculated from batches of 5 episodes. To reduce variance, returns were standardized on a per-episode basis and we maintained a running baseline, which is needed for numerical stability.

As expected the reward function played a crucial role during test runs. As rewards we implemented +0.02 for ball contact, +20 for scoring a point, −1 when the opponent scored and for winning a match a reward of +5000. These values were determined through trial runs in which different scales were compared. Without the small paddle hit reward the positive signal that the agent received was too small and it failed to learn. Also a negative point reward was necessary, as without it the agent remained passive and seemed to wait until a paddle hit happened by chance. Increasing the penalty for losing points (e.g., to −5) on the other hand caused training to collapse. The negative feedback was too strong.

Our hyperparameters were selected through small grid searches. Learning rates between 0.0005 and 0.003 were evaluated, with 0.001 and 0.0015 providing good convergence rates, but the latter performed more efficiently in longer runs. For the hidden layer we compared sizes of

64, 128, and 256. The smallest configuration tended to underfit slightly, whereas 256 units did not offer measurable benefits over 128. The batch size of five episodes represented the smallest value that still produced sufficiently stable gradient estimates without unduly increasing training time.

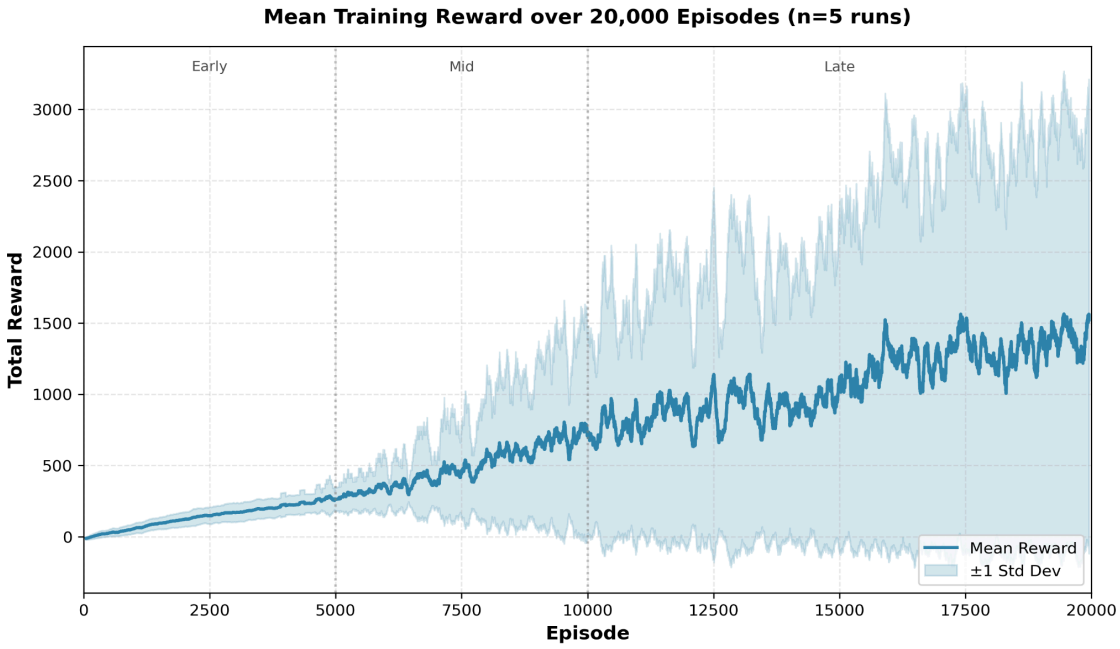
4.3. Results

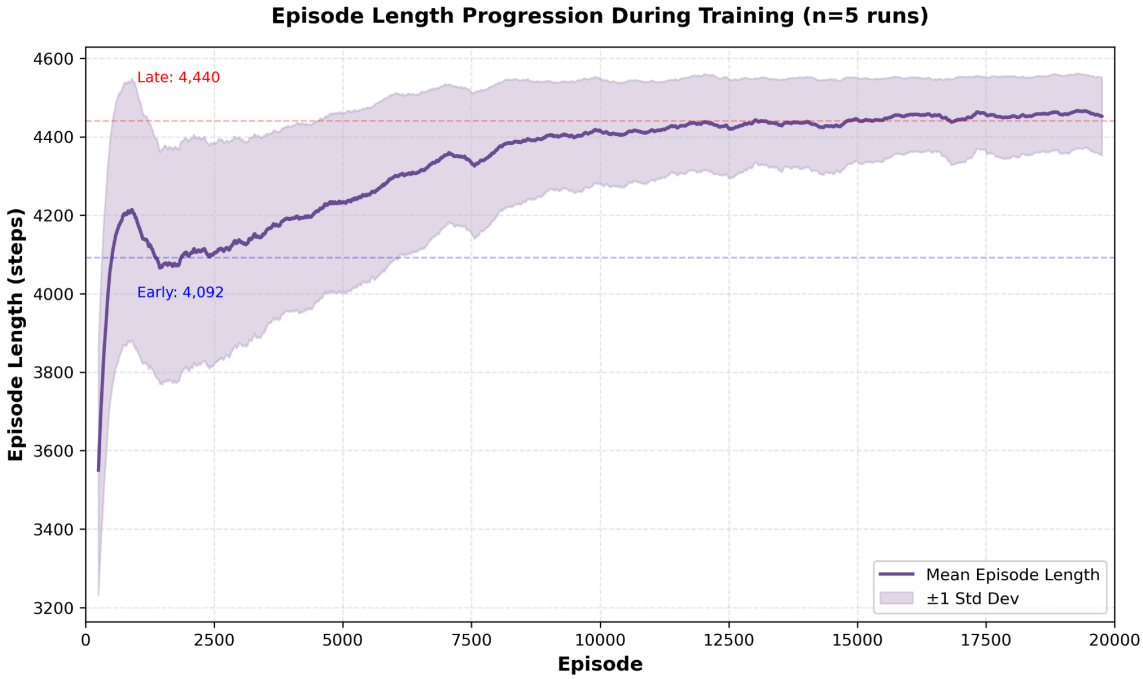
We trained five independent agents for 20,000 episodes each with a different random initialization. All reported results are based on the mean and standard deviation over these five runs. In the following analysis we focus on three aspects: the learning progress over time, the agent's ability to make use of the progressive angle mechanism, and the performance on the match level.

Training for 20,000 episodes took about six hours on a standard CPU (Intel i7, 3.6 GHz) without using a GPU. Due to the small size of our policy network (2,563 parameters), experiments and hyperparameter tuning could be carried out relatively quickly. Pixel-based approaches typically would require much larger networks and GPU acceleration. This did however not seem practical for this project scope.

To interpret the performance of the trained agent, we first evaluated simple baseline policies. Over 1,000 evaluation matches, both a purely random agent and an untrained policy network with randomly initialized weights achieved a win rate close to 0. While these agents occasionally scored single points due to the "loser serves" rule, they did not show any consistent paddle interaction. This shows that the game punishes random behavior and exploiting the progressive angles only starts to matter once the agent can consistently hit the ball. By comparison, our trained agent achieved an average score of $\mu = 15.7$ points. The agent successfully learned to handle the defensive aspects of the game, but it still remains far from reaching a competitive win rate of 50% against the rule-based opponent.

Training Progress: The learning curve showed rapid improvement in the first 5,000 episodes, with mean reward increasing from zero to approximately 140. Between episodes 5,000-10,000, the agent continued improving to around $\mu = 487$. After episode 10,000, performance continued to $\mu = 1,076$ ($\sigma = 265$). Episode duration increased by $\mu = 8.5\%$ from $\mu = 4,092$ steps early in training to $\mu = 4,440$ steps late in training. This indicates the agent learned to sustain longer rallies.



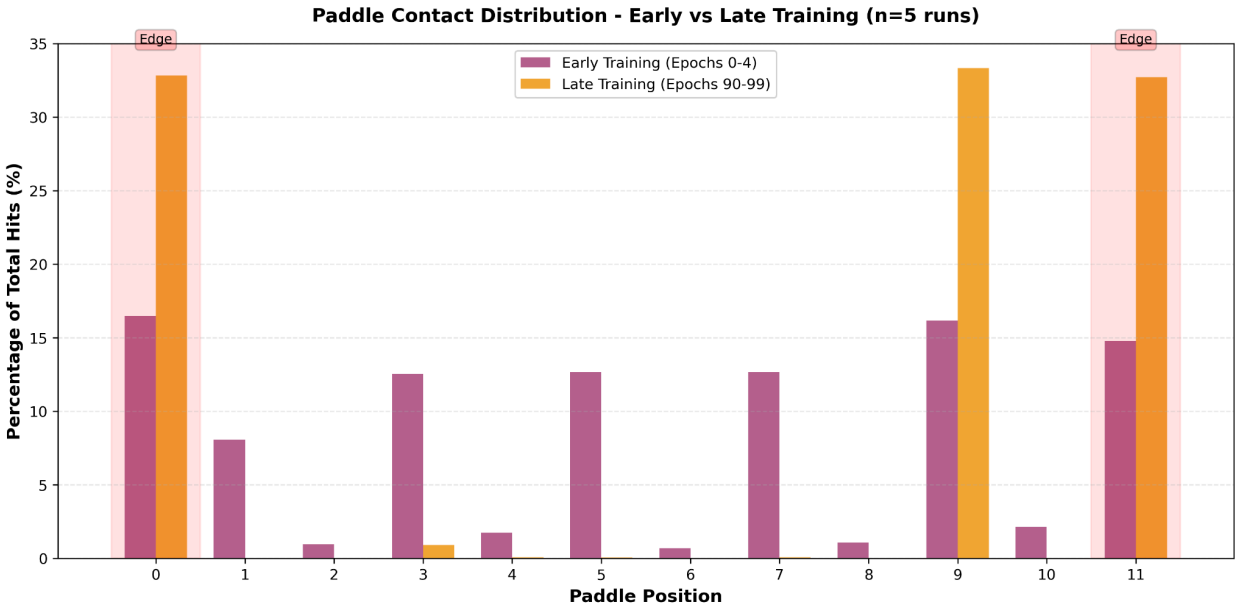


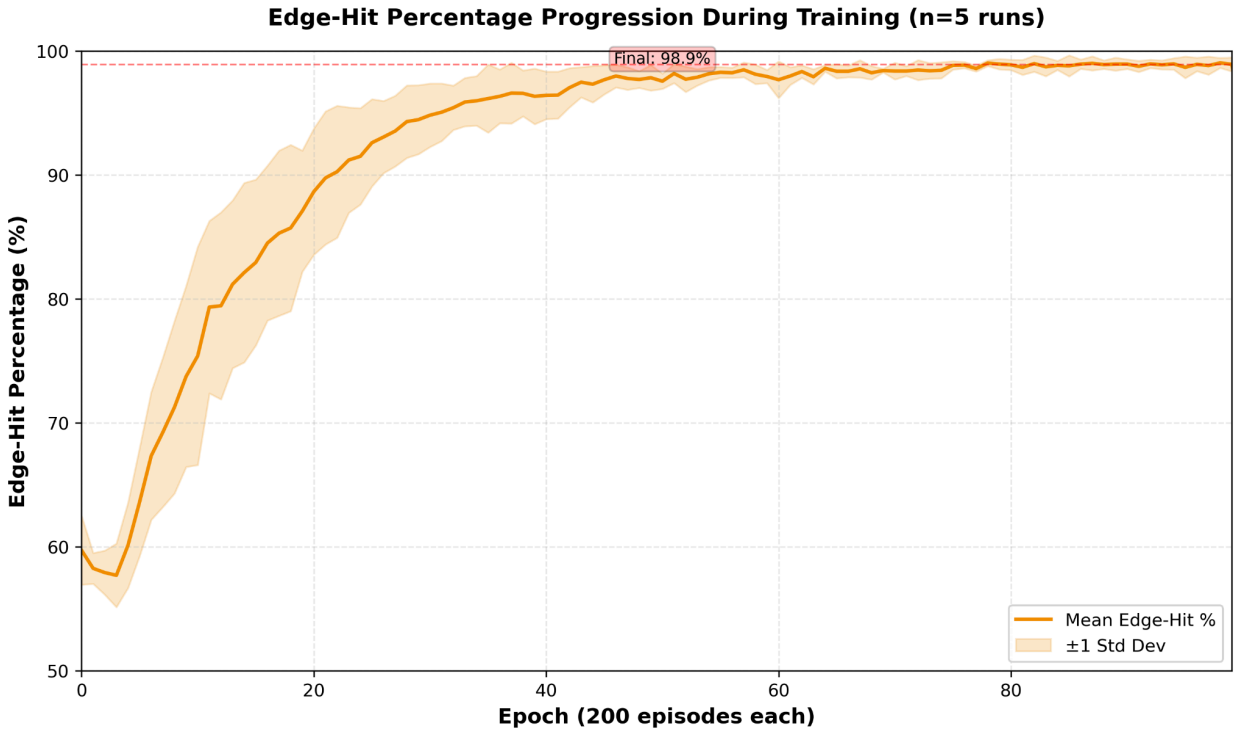
Win Rate and Scoring: The trained agent achieved a win rate of μ = 8.0% (σ = 3.36%) with an average score of μ = 15.7 (σ = 1.09) points per match versus μ = 20.9 (σ = 0.03) for the CPU opponent. This is a substantial improvement over the random baseline.

Trained Agent Performance Summary

Metric	Trained Agent (n=5)
Win Rate (%)	8.0 (σ=3.36)
Mean Score	15.7 (σ=1.09)
CPU Score	20.9 (σ=0.03)
Edge Hits - Early Training (%)	59.6 (σ=2.20)
Edge Hits - Late Training (%)	98.9 (σ=0.39)
Mean Episode Length (steps)	4329 (σ=36)

Edge-Hitting Emergence: Collision analysis across μ = 520,111 (σ = 20,384) paddle hits shows strong change of the agent's behavior over the course of the training. The paddle is divided into 12 discrete contact positions (0-11), where positions 0-2 and 9-11 represent the edge zones that produce the steepest bounce angles. During the first 1,000 episodes ball contacts were distributed relatively evenly on the paddle and edge hits made up for μ = 59.6% (σ = 2.2%) of all hits. By the final 1,000 episodes, edge hits had increased to μ = 98.9% (σ = 0.4%), which is an increase of μ = 39.3% (σ = 2.0%). The positions adjacent to the edges were only hit occasionally and central paddle hits were close to zero. This demonstrates that the agent successfully discovered and exploited the progressive angle mechanic and used edge hits to generate steep return angles that were difficult for the opponent to reach.





Strategic Analysis and Limitations: Even though the agent excelled at edge-hitting, it still lost most games. Manual observation shows that when the ball respawns in the center due to the "loser serves" rule, the agent makes no attempt to reach it. It is thus almost guaranteed that the CPU opponent wins back a point after the agent scored. This specific weakness explains why it consistently loses matches, even though its offensive skills were actually quite strong. So the net outcome is a 15.7 vs 20.9 score.

A likely explanation for this behaviour could be that REINFORCE updates based on episode returns and our features do not encode a game state like "serve incoming". Our agent struggles with such context-dependent policies. Meaning the agent cannot distinguish rally context from serve context and failed to learn a good policy (like positioning in the center), after a paddle hit.

5. Discussion

5.1. Success in Exploration and Micro-Skill Acquisition

We tested whether reward shaping alone could teach a simple REINFORCE agent to use the "progressive angle" mechanic and play well. Our results allow us to clearly answer this question. Overall, results show we largely solved the usual problems of sparse rewards and figuring out which moves matter (credit assignment). This is backed up by data. Across five different runs, learning was consistent, shown by the low variance in the final reward. This proves that our designed reward structure stabilized the training, which is usually very sensitive to random luck

in REINFORCE. More importantly, the agent actually learned the specific skill we wanted. By the end of the training it had learned to exploit the progressive angle system. The observed shift was clear: An average edge-contact rate of μ = 98.9% shows the agent learned to aim for the edges to let the ball return at sharp angles. Since this is the best possible strategy in the game, our results confirm that reward shaping can successfully guide the agent to find complex, high-reward strategies.

5.2. Failure in Context-Dependent Policy and Mastery

Despite mastering the offensive micro-skill, the agents exhibited a robust failure to achieve match-level competency, consistently losing with a final score of ~16 to 21. This indicates that the problem does not come from the reward function. The real issue lies in the agent's architecture and how it sees (or does not see) the game state. The big issue we kept seeing was that after the CPU scored, our agent just remained passive instead of preparing for an incoming serve. A human player would position himself in the center to react most flexibly. Our agent however remained passive. Why? Because the agent failed to tell the difference between two different situations: (1) normal rally play and (2) the loser-serves moment right after scoring a point. Since we only gave one policy and trained it on total episode score, it naturally only focused on what resulted in the most rewards in the long run (edge-hits). Strategic repositioning in the center however gives no immediate reward, so the agent never bothered learning this behavior. This shows a bigger challenge in reinforcement learning: just maximizing the total reward at the end of an episode doesn't automatically give you good behavior in every short-term situation, especially when important context (like "a random serve is coming") is not included in the state the agent sees. In short, the agent got really good at one thing (edge-hitting during rallies) but failed to learn to switch to a defensive strategy when the context changed.

5.3. Comparison to Prior Work

While modern algorithms like PPO and A3C use architectural modifications (e.g., critics, constrained updates) to reduce variance and improve sample efficiency, our results show that the limitations of vanilla REINFORCE extend beyond instability and slow convergence. Here, the limitation is rooted in the agent's difficulty in learning distinct behavioral modes from a single, high-variance policy gradient signal, even when features are engineered and rewards are shaped. The challenge is therefore not just one of learning speed, but of learning complexity. The observed failure strongly suggests that for environments with clear phase transitions or required contextual switches, a simple policy gradient agent either requires a more informative

feature space or necessitates an architecture, like an Actor-Critic model, capable of using an estimated value function to reduce variance and guide learning across disparate contexts.

6. Conclusion and Outlook

6.1. Conclusion

This paper documented whether a feature-based vanilla REINFORCE agent could learn to exploit the complex progressive angle mechanics in a custom Pong environment. We tried this using reward shaping, through five independent training runs. The results showed that on the one hand our approach was a success in guiding exploration, but it was a failure in learning a complete, winning strategy. The primary finding is that our structured, static reward function successfully solved the severe credit assignment problem, which is inherent to sparse-reward, episodic policy gradient methods. The agent consistently learned to specialize in playing offense. This was confirmed by how stable the results were (low variance, $\sigma = 85$ on final reward) and the high percentage of edge hits ($\mu = 98.9\%$) it achieved late in training. Those findings prove our initial idea that using different reward levels can successfully guide the agent to explore and find complex, high-reward skills.

However, the agent failed in its attempts to win matches consistently (mean win rate of $\mu = 8\%$), which shows that reward shaping alone was not enough for it to truly master the game mechanics. Our analysis pointed to one main problem: the agent couldn't learn a separate policy for the serve-return context, which is why it kept losing points. This weakness means the feature representation itself was limited, as it didn't include a signal for "serve incoming." Also, it was simply difficult for a basic REINFORCE method to reliably switch behaviors when the game context changed, especially with noisy, delayed rewards.

In summary, reward shaping worked great for helping the agent explore and learn a specific micro skill (edge-hitting), but it didn't solve the main problem of whether the policy was flexible enough to fully master the game.

6.2. Outlook

Based on our findings, future improvements of our algorithm should focus on two main issues:

Feature Augmentation and Contextual Policy: We should evaluate how the agent reacts to a new feature in the state vector that represents states of the game. A binary feature indicating “is_serve_incoming” would very likely help the agent read and adapt to the “loser serves”-rule.

Algorithmic Enhancement for Stability: Since our game setting requires relatively long runs and sparse rewards, we should investigate using an Actor-Critic architecture (like A2C). An AC

approach uses a learned value function, called the Critic, to lower the variance in the policy gradient estimate. This stability could potentially help the agent learn across different game contexts much better than the high-variance Monte Carlo method we used (REINFORCE).

7. Bibliography

Bellemare, M. G., Naddaf, Y., Veness, J., & Bowling, M. (2013). The Arcade Learning Environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47, 253–279. https://doi.org/10.1613/jair.3912

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Lapan, M. (2018). Deep reinforcement learning hands-on: Apply modern RL methods, with deep Q-networks, value iteration, policy gradients, TRPO, AlphaGo Zero and more. Packt Publishing.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing Atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602. https://arxiv.org/abs/1312.5602

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. https://doi.org/10.1038/nature14236

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning. In Proceedings of the 33rd International Conference on Machine Learning (pp. 1928–1937). PMLR.

Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations: Theory and application to reward shaping. In Proceedings of the Sixteenth International Conference on Machine Learning (pp. 278–287). Morgan Kaufmann.

Schulman, J., Levine, S., Moritz, P., Jordan, M. I., & Abbeel, P. (2015a). Trust region policy optimization. In Proceedings of the 32nd International Conference on Machine Learning (pp. 1889–1897). PMLR.

Schulman, J., Moritz, P., Levine, S., Jordan, M. I., & Abbeel, P. (2015b). High-dimensional continuous control using generalized advantage estimation. In Proceedings of the 4th International Conference on Learning Representations (ICLR).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347. https://arxiv.org/abs/1707.06347

Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (1999). Policy gradient methods for reinforcement learning with function approximation. In Proceedings of the 12th International Conference on Neural Information Processing Systems (pp. 1057–1063). MIT Press.

Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., & de Freitas, N. (2016). Dueling network architectures for deep reinforcement learning. In Proceedings of the 33rd International Conference on Machine Learning (pp. 1995–2003). PMLR.

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning, 8(3–4), 229–256. https://doi.org/10.1007/BF00992696

8. Appendices

8.1. Hyperparameters and Reward Shaping

Hyperparameter Configuration

Parameter	Value	Description
Input Layer Size	16	Dimensionality of the feature vector
Hidden Layer Size	128	Number of neurons in the single hidden layer
Optimizer	RMSprop	Optimization algorithm for gradient updates
Learning Rate (α)	0.0015	Step size for weight updates
Discount Factor (γ)	0.99	Weighting of future vs. immediate rewards
Batch Size	5 episodes	Episodes collected before a policy update
Maximum Episodes	20	Total duration of the training phase

Reward Shaping Configuration

Event	Reward Value	Purpose
Paddle-Ball Contact	0.02	Encourages defensive positioning (Stage 1)
Scoring a Point	20.0	Encourages offensive goal-seeking (Stage 2)
Opponent Scores	-1.0	Penalizes defensive failure
Winning the Match	5000.0	Encourages exploitation of angle mechanics (Stage 3)
Losing the Match	0.0	Neutral terminal state

8.2. Source Code (GitHub Link)

https://github.com/kormuch/Reinforcement-Learning/tree/main/RL_Pong_V1.0